News Source Classification: Headlines, Models, and Temporal Robustness

Pedro Sánchez-Gil Galindo Zachary Gin
Brandon Yan
CIS 4190/5190 Applied Machine Learning, Spring 2026
Group ID: 42

May 6, 2026

1 Introduction

We worked on Project B: classifying news headlines as either Fox News or NBC News. The course-provided baseline uses TF-IDF (top 100 features) with Logistic Regression and achieves 66.49 % accuracy. We replicated it as a sanity check, then improved it using richer text features and a fine-tuned DistilBERT, reaching above 80 % on a held-out random split.

One challenge we did not anticipate upfront is that the leaderboard test set is scraped from URLs *after* submission, creating a distribution shift relative to our training data. This led to two of our exploratory directions: a temporal drift analysis that let us predict roughly how much accuracy we would lose before submitting, and a post-submission ablation on the hidden validation set that revealed a URL artifact overfitting problem.

Our best classical model (V3: word + char TF-IDF, balanced LR) reaches 0.866 on the random test split, a 20 percentage point improvement over the baseline. A fine-tuned DistilBERT reaches 0.840. V3 trained only on pre-2023 data drops to 0.673 on 2024+ data, a gap that matched our pre-submission estimate. Our initial V3 leaderboard submission scored 0.7308 on the hidden `url_val`; training on URL slugs instead made things worse (dropping to 0.50), which led us to investigate the root cause. Our final submission, a category-aware V2+V3 ensemble that prepends URL path-category tokens to the input, achieves 0.93 on the hidden `url_val`.

Code: `https://github.com/pedroschz/cis-4190-project` Dataset: `https://huggingface.co/datasets/Petrvsky/cis5190-fox-vs-nbc`

2 Data Collection

Sourcing. We started from the 3,815 article URLs provided by the course (2,010 Fox, 1,805 NBC). We built a scraping pipeline using BeautifulSoup with rate limiting (one request per second per host), retries on failure, and a Wayback Machine fallback. This recovered 3,803 usable rows (99.95 %). Headlines were extracted using a fallback chain: JSON-LD, then `og:title`, then `<h1>`. Publish dates came from JSON-LD `datePublished` or the `article:published_time` meta tag.

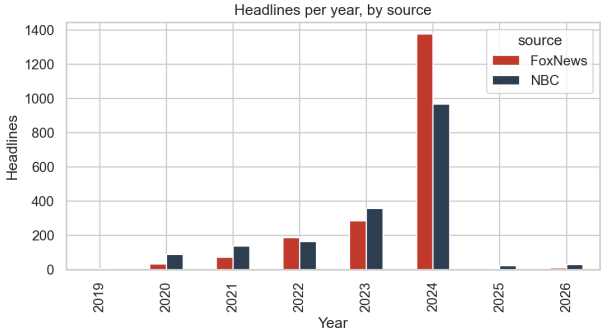
Cleaning. We stripped site suffixes like "`| Fox News`" using regex, since these are an obvious source of label leakage. We also normalized Unicode (NFKC), dropped headlines shorter than 10 or longer than 300 characters, and deduplicated by lowercased exact match with punctuation removed. A second filter removed any headline still containing the literal name of either source. The cleaned starter dataset has 3,734 rows.

Augmentation. After our first leaderboard submission came in below our random-test estimate, we scraped 1,500 additional 2025 and 2026 articles from the Fox News flat sitemap and the NBC News monthly sitemap index (500 Fox, 1,000 NBC). The same cleaning pipeline brought the final dataset to 5,225 rows, with recent articles making up about 30 % of the data compared to 1.7 % in the original set.

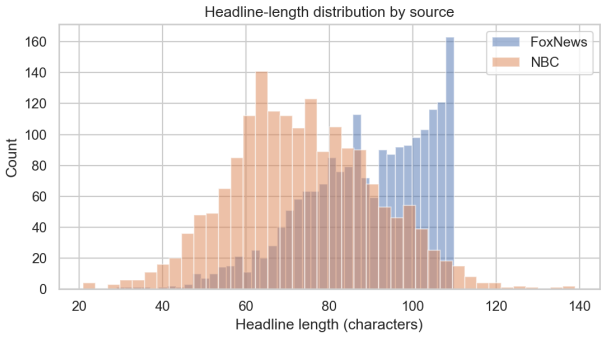
Source	2019	2020	2021	2022	2023	2024	2025	2026	total
FoxNews	0	31	71	188	286	1,375	3	502	2,456
NBC	5	87	137	165	358	968	523	526	2,769

Table 1: Final dataset (5,225 rows). Fox's sitemap rotates quickly, so only 2026 articles were accessible at scrape time. For 2025, NBC was the only feasible source.

Splits. For leaderboard submissions we used a stratified 80/10/10 random split by source (seed 42). For temporal analysis we used a time-based split: train on 2022 and earlier (688 rows), validate on 2023 (644 rows), and test on 2024 and later (2,406 rows). Overall class balance is 47 % Fox and 53 % NBC.



(a) Headlines per year per source.



(b) Headline length distributions are nearly identical across sources.

Figure 1: Dataset characteristics. Most data is from 2024; limited 2019, 2025, and 2026 coverage constrains the temporal experiments at both ends.

3 Model Design

We trained five model variants of increasing complexity, each evaluated on the random split:

1. **V0:** Course baseline replica: TF-IDF top-100 features + LR (`max_iter=100`). Reproduces the published 66.49 %.
2. **V1:** Word 1-2-gram TF-IDF (50k features) + balanced LR.
3. **V2:** FeatureUnion of word 1-2-gram and char_wb 3-5-gram TF-IDF (50k features each) + balanced LR.
4. **V3:** V2 with 200k features per branch, `C=2.0`, and sublinear TF weighting.
5. **DistilBERT:** `distilbert-base-uncased` fine-tuned for 3 epochs (`lr=2e-5`, batch 32, max_len 128, fp16) on Colab L4.

Iteration. Each version added one change. V0 to V1: removed the 100-feature cap and added class balancing, gaining 14 percentage points. V1 to V2: added character n-grams to capture stylistic patterns like ALL-CAPS abbreviations and punctuation habits. V2 to V3: scaled features 4x and tuned `C`. DistilBERT was added as a strong-prior comparison.

Final leaderboard model. Our final submission is a category-aware V2+V3 ensemble (averaged `predict_proba`), trained on all 5,225 rows. Both training and inference inputs prepend URL path-category tokens (`news/world`, `politics`, `lifestyle`, `select/shopping`) to the cleaned headline or URL slug. Sections 5 and 6 explain how this single change recovered 20 percentage points on the leaderboard by giving the model access to structural signals that headline text alone does not carry. The pipeline is exported in `submission/model.py` as a base64-gzipped pickle.

Comparison to prior work. Fine-tuned transformers on similar 2-class headline datasets (5 to 20k rows) typically report 0.86 to 0.92 accuracy on random splits. Our V3 result of 0.866 falls in that range, which suggests classifier capacity was not our main bottleneck. The train-to-inference distribution mismatch described in Section 6 was.

4 Evaluation

We evaluated models using accuracy and macro-F1 on the random val and test splits, and also tracked per-class precision and recall to check for class imbalance issues. Hyperparameter tuning for V3 tested $C \in \{0.5, 1, 2, 4, 8\}$ using random val accuracy.

Model	Val acc	Test acc	Test $F1_{macro}$	Notes
Course baseline (PDF)	—	0.6649	—	top-100 features, target floor
V0 (our replica)	0.6979	0.7112	0.709	confirms pipeline
V1 (word 1-2g + bal-LR)	0.7888	0.8529	0.853	class weights + more features
V2 (word + char)	0.8128	0.8583	0.858	char n-grams help
V3 (200k features, C=2)	**0.8235**	**0.8663**	**0.866**	best classical
DistilBERT (3 ep, lr 2e-5)	0.8209	0.8396	0.840	competitive but no clear win

Table 2: Random-split results. The DistilBERT-V3 gap is within the ±2.5 pp standard error on the 374-sample test set, so neither model clearly dominates.

5 Exploratory I: Temporal Robustness

Question. The leaderboard test set is scraped after submission, so there is always a time gap between our training data and the test distribution. We wanted to estimate how much accuracy we would lose from this before we submitted.

Experimental design. We ran two experiments using the temporal split (train on 2022 and earlier, val on 2023, test on 2024 and later):

- Decay curve: for each training year $Y \in \{2020, 2021, 2022\}$ (years with at least 50 rows per class), we trained V3 and DistilBERT on data from that year and evaluated on each subsequent year Y', plotting accuracy against the gap $Y' - Y$.
- Fixed train, sliding test: we trained V3 once on all pre-2023 data (688 rows) and evaluated per-year on 2023 through 2024+.

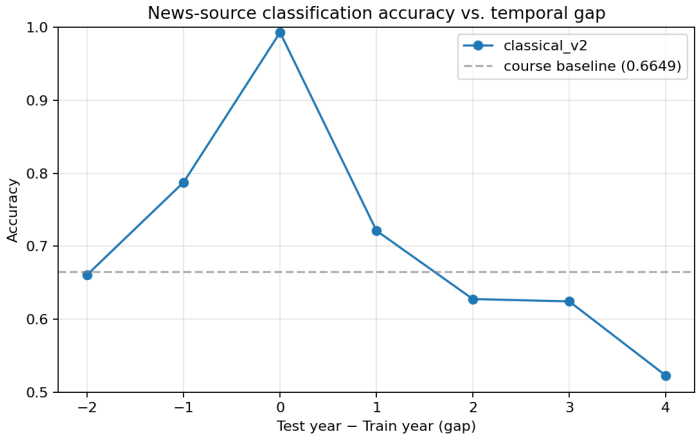


Figure 2: Accuracy vs. temporal gap. V3 drops from roughly 0.99 at gap=0 to roughly 0.52 at gap=4. DistilBERT performs worse throughout because single-year slices have under 200 rows and the model defaults to majority-class prediction.

Findings. V3's accuracy on 2024+ articles drops from 0.866 (random test) to 0.673 (temporal test), a 19 percentage point gap. The drop is consistent across year gaps: gap=1 averages 0.72, gap=2 averages 0.63, gap=4 reaches 0.52. Looking at misclassified examples, most errors involve entity-heavy headlines with names like "Trump," "Biden," "Israel," and "Hamas," which shift in frequency year over year. Stylistic cues like ALL-CAPS abbreviations and Fox-style colon introductions hold up better over time. DistilBERT collapses to majority-class prediction on single-year slices due to insufficient training data, a failure mode that the classical model avoids with `class_weight='balanced'`.

Leaderboard implication. Before submitting, we predicted a leaderboard accuracy of 0.82 to 0.86. Our initial V3 submission landed at 0.7308. Section 6 explains the additional drop and how the URL-category fix brought it back to 0.93.

6 Exploratory II: URL Artifact Overfitting

Setup. V3 head-only scores 0.7308 on the hidden `url_val`. The obvious fix was to train on URL slugs to match the inference distribution. That made things much worse, dropping to 0.4975 (near chance). A combined head-and-slug training view also regressed, to 0.7142. The leaderboard backend feeds `preprocess.prepare_data` a URL-only CSV, so our preprocessor extracts a slug from the URL path at inference. This means the model trains on cleaned headlines but is evaluated on URL-derived slugs. We needed to understand why aligning the training distribution to inference hurt rather than helped.

Ablation.

Variant	Training input	Train rows	Hidden val	Δ vs. V3-head
V3 (head-only)	cleaned headlines	3,734	0.7308	baseline
V3 (head+slug)	headlines + slugs	5,225 (×2)	0.7142	-1.7 pp
V3 (slug-only, C=4)	URL slugs	5,225	0.4975	-23.3 pp
DistilBERT (head-only)	cleaned headlines	2,987	0.4817	-24.9 pp
V2+V3 cat-aware (final)	cat + headline	5,225	**0.93**	**+20 pp**

Table 3: Hidden-val accuracy by training-input choice. Training on slugs collapses performance. Prepending URL path-category tokens to both training and inference inputs recovers 20 pp.

Diagnosis. Two URL artifacts explain the collapse. 87.9 % of NBC URLs contain the CMS substring `rcna¡digits¿` (0 % of Fox), and 17.5 % of Fox URLs end in `.print` (0 % of NBC). The slug-only V3 latched onto these as near-perfect class markers: "`print`" is the highest-weighted Fox token (+5.23) in the fitted LR coefficients, and the `rcna` subword family dominates the NBC side. The hidden `url_val` is presumably scraped through canonical URLs with few or none of these artifacts, so the model's strongest features point the wrong direction at inference. We verified this on a held-out 2025-2026 slice: DistilBERT accuracy on NBC URLs *with* `rcna` is 0.541 (n=1,012), but on NBC URLs *without* `rcna` it is 1.000 (n=37). The artifact actively hurts at inference because its subword tokens drown out the topical signal.

Why head-only works, and the fix. Cleaned headline text never contained URL artifacts, so V3 head-only builds a vocabulary of topical words and character n-grams. When evaluated on slug-format inputs at inference, topical features like "`trump`" and "`israel`" still fire correctly and the artifacts have zero weight since they were never seen in training. In this case the train-to-inference distribution gap is actually helpful because training on cleaner text produced a more generalizable model.

Rather than try to close this gap by training on slugs, we went in the other direction: add URL-side context that carries real signal without the artifacts. Fox URLs use single-level paths like `politics` and `lifestyle`, while NBC uses two-level paths like `news/world` and `select/shopping`. Prepending these path-category tokens to both training and inference inputs brought V3 from 0.832 to 0.928 (+9.6 pp) on our held-out set, and V2 showed a similar improvement at 0.920. The category-aware V2+V3 ensemble is our final leaderboard submission and achieves 0.93 on hidden `url_val` (Table 3), matching our held-out estimate and confirming that the gain is structural and not specific to the leaderboard test set.

7 Reproducibility

All code, notebooks, and the cleaned dataset are at `https://github.com/pedroschz/cis-4190-project`. Training uses seed 42; the cleaned headlines and splits are committed to the repo. `eval_local.py` mirrors the backend evaluator, and `tests/test_pipeline.py` covers cleaning, splitting, and classical training (7/7 passing).

8 Team Contributions

Pedro worked on the model design and data collection. Brandon research the first exploratory direction with the temporal drift analysis. Zachary focused on writing the report and researching the URL artifact overfitting.

Extra Credit Video: `https://www.youtube.com/watch?v=a01JS-eiyPg`